4.3 Composer initialization and script aliases

Chapter 20 introduces Composer as the tool that manages dependencies and generates loading code. Begin at the repository root, alongside `src` and `public`:

```
php --version
composer --version
```

Use the course PHP 8.4 environment. If Composer is missing, follow the official installation instructions, open a new terminal, and repeat the checks. Composer needs a working CLI PHP installation.

Walk through composer init

If the repository already has `composer.json`, read it and merge this lesson's settings. Otherwise run:

```
composer init
```

Prompt wording can vary with Composer releases. Explain each choice as you enter it:

Prompt	Suggested response	Reason
Package name	`student/cis334-app`	Lowercase vendor/project identifier; students may replace student
Description	`CIS 334 application`	Human-readable purpose
Author	Your details, or `n` to skip	Metadata; not an account registration
Minimum stability	Press Enter	Keep stable dependencies by default
Package type	`project`	This is an application, rather than a reusable library
License	`proprietary` for this private classroom sample	Use an actual chosen license if the project has one
Define dependencies interactively?	`no`	Install the two packages together in 4.6
Define dev dependencies interactively?	`no`	None needed for this lesson
Add PSR-4 autoload mapping?	`n` if offered	We will explicitly configure App in 4.5
Confirm generation?	`yes`	Save the reviewed configuration
Add vendor to .gitignore?	`yes` if offered	Dependencies can be reinstalled

Do not accept `library` as the type for this application. Init writes configuration; it does not publish the project or create a framework application. If an init version generates a namespace from the package name, replace that generated mapping with App in 4.5.

Read and edit JSON

Open the resulting file. JSON uses double-quoted keys and strings, commas between members, no comments, and no trailing commas. Preserve author and other useful existing fields. Add these two top-level members (the object below is a **merge example**, not an instruction to erase other settings):

```
{
    "require": {
        "php": "^8.4"
    },
    "scripts": {
        "hello": "echo Hello class 4",
        "demo": "@php public/tutorials/4-1-namespaces.php"
    }
}
```

The PHP constraint describes this project's supported runtime. `@php` uses the PHP executable running Composer. Demonstrate the aliases:

```
composer validate
composer hello
composer demo
```

Expect `Hello class 4` and then the two User output lines from 4.1. Composer may also print the command being executed. `composer run-script demo` is the explicit form of `composer demo`.

References: init command, package type, and Composer scripts.